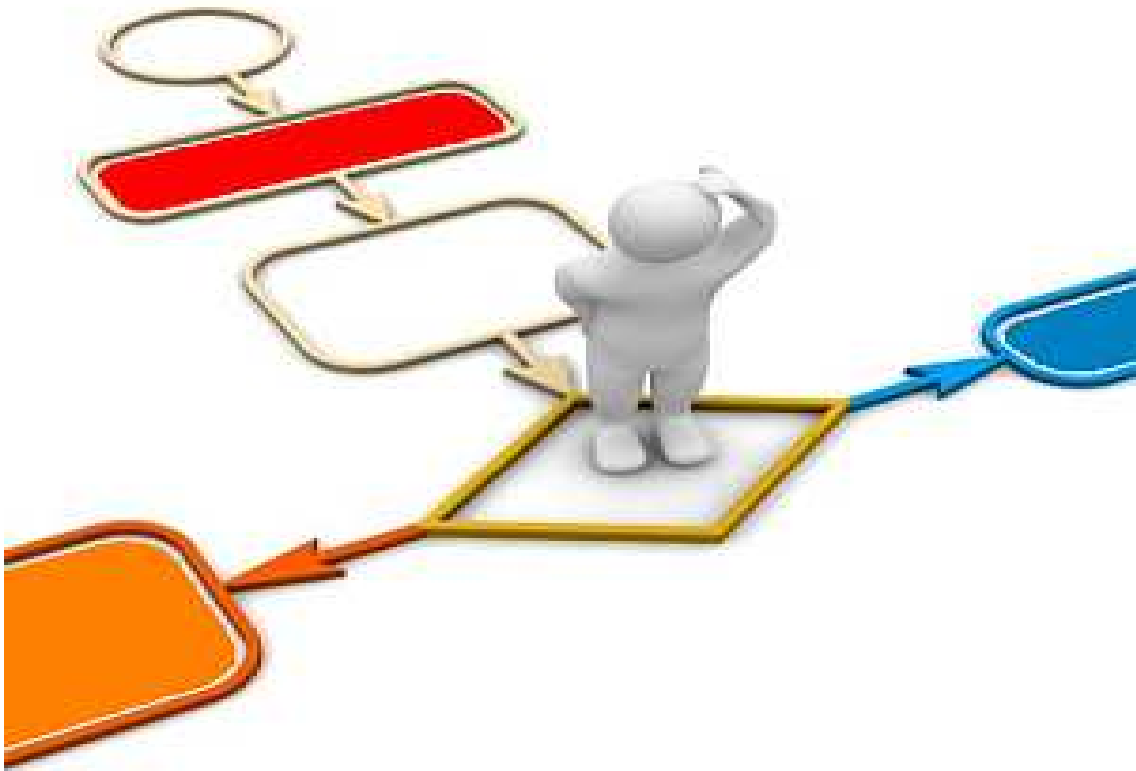


UNITÉ D'APPRENTISSAGE 5 :

UTILISER LES STRUCTURES DE CONTROLE

Compétences visées :

- Développement du raisonnement logique pour résoudre les problèmes
- Initiation au raisonnement logique ;
- Résolution numérique d'un problème simple de calcul ;
- Automatisation des calculs répétitifs



Leçon 12 : Les éléments d'un algorithme

Leçon 13 : Structures de contrôle

Leçon 14 : Exécution d'un algorithme

Leçon 15 : Organigramme

OBJECTIFS PÉDAGOGIQUES :

- ✓ Identifier les éléments d'un algorithme ;

CONTROLE DE PRESREQUIS :

1. Définir algorithme
2. Connaître les qualités d'un bon algorithme
3. Connaître les étapes de résolution d'un problème

SITUATION PROBLEME :

Votre petit frère dispose de deux bouteilles de jus de fruits:

- Dans la première bouteille nommée A se trouve le jus d'orange;
- Dans la deuxième bouteille nommée O se trouve le jus d'ananas.

Il souhaiterait que le jus d'ananas se retrouve dans la bouteille nommée A et le jus d'orange dans la bouteille nommée O. Ne pouvant pas résoudre tout seul son problème, il fait appel à vous.

Consignes :

1. Identifier deux contenants dans le problème que pose votre petit frère, comment les appelle-t-on en algorithmique ? (**Reponse attendue** : la bouteille nommée A et la bouteille nommée O, on les appelle les variables en algorithmiques)
2. Quel élément votre petit frère a-t-il besoin nécessairement pour effectuer cet échange ? (**Reponse attendue** : il a besoin d'une troisième bouteille vide nommée B pour effectuer l'échange...)
3. Tenant compte de l'élément de réponse de la question 2, donner une procédure à votre petit frère afin qu'il réalise son souhait (**Reponse attendue** :
 - ❖ Dans la bouteille vide nommée B verser le jus d'ananas contenu de la bouteille nommée O ;
 - ❖ Dans la bouteille nommée O verser le jus d'orange contenu de la bouteille nommée A ;
 - ❖ Et enfin dans la bouteille nommée A verser le jus d'ananas contenu de la bouteille nommée B.).
4. Comment appelle-t-on les étapes de résolution d'un problème ? (**Reponse attendue** : l'algorithme)

RESUME

DÉFINITIONS

Algorithme est une suite finie et ordonnée d'actions permettant de résoudre un problème donné.

Algorithmique est la science qui étudie les algorithmes.

Variable : c'est un objet dont le contenu peut changer au cours de l'exécution de l'algorithme.

Constante : c'est un objet dont le contenu reste invariant lors de l'exécution de l'algorithme

PARTIES D'UN ALGORITHME

Un algorithme est composé de trois grande partie :

- **L'entête** : constitué du nom de l'algorithme
- La **partie déclarative** : elle comporte la déclaration des variables et constantes.
- Le **corps** : il est constitué de l'ensemble de toutes les actions qui seront traitées dans l'algorithme

Le nombre d'actions se décompte dans le corps de l'algorithme

OBJETS MANIPULES EN ALGORITHMIQUE

Les objets les plus utilisés en algorithme sont : Les **variables** ; les **constantes**. Avant d'utiliser un objet dans un algorithme, il faut au préalable le déclarer. La syntaxe de déclaration dépend de la nature de l'objet :

- ✓ **Déclaration d'objets variables**

Syntaxe : **Var** ou **Variable** **Identifiant** : **Type**

Exemple : **Var** note, P, R : Entier ;

- ✓ **Déclaration d'objets constants**

Syntaxe : **Const** ou **Constante** **Identifiant** = **Valeur_initiale** ;

Exemple : **Const** Pi = 3,14;

OPÉRATEURS PRIMITIFS

Un **opérateur** est un symbole permettant d'élaborer une expression en vue d'effectuer un calcul ou une comparaison. Ainsi, on distingue :

- ✓ **Les opérateurs arithmétiques**

En plus des opérations arithmétiques de base (addition, soustraction, multiplication et division), le tableau suivant illustre les différents opérateurs arithmétiques.

Opérateur	Signification	<u>Exemple</u> :
Div.	Division entière	11 DIV 4 = 2
^	Puissance	2^3 = 8
MOD	Reste de la division entière	11 MOD 4= 3

- ✓ **Les opérateurs relationnels**

Opérateur	Signification	Opérateur	Signification
<	Inférieur	>=	Supérieur ou égale
<=	Inférieur ou égale	=	Égale
>	Supérieur	≠ ou <>	Différent

✓ Les opérateurs logiques

Ils sont utilisés pour relier des expressions logiques ou des identificateurs de type booléen. Pour ce qui est de l'algorithmique, une expression logique comporte un ou plusieurs symboles suivants : le « ou » logique (OU) ; le « et » logique (ET) ; le « non » logique (NON).

Opérateur	Signification	Exemple
ET	« et » logique	$(15 > 5) \text{ ET } (6 < 3) = \text{FAUX}.....$
OU	« ou » logique	$(15 > 5) \text{ OU } (2 > 5) = \text{VRAI}.....$
NON	« non » logique	$\text{NON}((15 > 5) \text{ OU } (2 > 5)) =\text{VRAI}.....$

INSTRUCTIONS SIMPLE EN ALGORITHMIQUE

Une **instruction** est une opération élémentaire, interprétée et exécutée par un processeur.

Les instructions simples les plus utilisées en algorithme sont : la **lecture**, l'**écriture** et l'**affectation**.

✓ Instruction d'affectation

Cette instruction (matérialisée par le symbole $\leftarrow$) permet d'attribuer une valeur ou le résultat d'une expression à une variable.

Syntaxe : **Nom_de_la_variable** $\leftarrow$ **valeur** | **expression**;

Exemple 1 : $\text{Somme} \leftarrow A+B$, on lit : à Somme j'affecte A plus B (L'objet Somme contient la somme des contenus des objets A et B)

$A \leftarrow 25$; on lit : à A j'affecte 25 (l'objet A contient la valeur 25)

NB : *Lorsqu'on affecte une valeur à un objet qui en contient déjà une, l'ancienne valeur de l'objet est détruite au profit de la nouvelle.*

Exemple 2 :

$A \leftarrow 4$ (A vaut 5)

$B \leftarrow 4$ (B vaut 4)

$A \leftarrow A+B$ (A vaut 9 soit $5+4$, l'ancienne valeur a été détruite).

✓ Instruction de lecture ou de saisie

Cette instruction permet d'entrer une donnée à partir du clavier. La machine attend que l'utilisateur tape une valeur au clavier.

Mot clé : **saisir** ou **Lire**

Syntaxe : **lire (liste des variables)** (nom variable étant une variable d'un type déclaré à l'avance).

NB : Les variables sont séparées entre elles par des virgules.

Exemple : **Lire (a, b)** ; ou bien **Lire(a)** ; **Lire(b)** ;

Ces instructions permettent de stocker dans les variables a et b les valeurs saisies au clavier.

✓ Instruction d'écriture ou d'affichage

Cette instruction permet d'effectuer l'affichage à l'écran.

Mot clé : Écrire ou afficher

Syntaxe : Écrire (liste des objets) ; Les objets sont séparés entre eux par des virgules.

Exemple 1 : Écrire ("Mon Premier Algo")

Cette instruction affiche la chaîne « **Mon Premier Algo** » à l'écran.

Exemple 2 : Var note1 : Entier ;

note1 ← 13 ;

Écrire (note1) ;

Cette instruction affiche la valeur contenue dans l'objet note1. (C'est-à-dire 13)

A côté de ces instructions de base, nous avons également **L'incrément** et **La décrémentation**.

✓ **L'incrément et La décrémentation**

Elles permettent respectivement d'ajouter ou de diminuer d'une **unité** le **contenu** d'une **variable**.

Exemples : $i \leftarrow i+1$; et $j \leftarrow j-1$;

SITUATION D'INTEGRATION :

Vous désirez écrire un algorithme pour le calcul de la somme de deux nombres.

1. Définir algorithme
2. Donner les parties d'un algorithme.
3. Donner la différence entre variable et constante.
4. Donner la syntaxe de déclaration d'une variable et celle d'une constante.
5. Quelles sont les instructions de base utilisées en algorithmique.

REINVESTISSEMENT

1. Écrire un algorithme qui calcule la surface d'un cercle.
2. Ecrire un algorithme qui demande à l'utilisateur son nom puis lui affiche un message pour lui dire bonjour.

OBJECTIFS PEDAGOGIQUES :

- ✓ Identifier les structures de contrôle

CONTROLE DE PREREQUIS :

1. Donner la structure minimale d'un algorithme
2. Utiliser les instructions d'écriture, d'affichage et d'affectation pour écrire les algorithmes.

SITUATION PROBLEME :

Vous souhaitez un algorithme permettant de résoudre une équation à coefficients réels de la forme $ax+b = 0$ (a et b seront entrés au clavier).

Consignes :

1. Lister les différentes structure de contrôle. (**Réponse attendue** : structures alternatives ou conditionnelles, les structures itératives ou boucles).
2. Quelle structure convient-il ici ? (**Réponse attendue** : structure alternative car il ya une condition sur le coefficient a ($a=0$ ou $a\neq 0$))
3. Donner sa syntaxe d'utilisation (**Réponse attendue**

Si (condition) alors
 (actions)
 sinon (actions)

RESUME

Les opérations élémentaires relatives à la résolution d'un problème peuvent, en fonction de leur enchaînement, être organisées suivant quatre familles de structures algorithmiques fondamentales : Structures linéaires, structures alternatives, structures de choix et structure itératives (ou répétitives).

STRUCTURES CONDITIONNELLES

Une structure conditionnelle est dite à forme alternative lorsque le traitement dépend d'une condition à deux états:

- Si la condition est évaluée à vrai, le premier traitement est exécuter;
- Si la condition est évaluée à faux, le 2^{ème} traitement est exécuté.

Une structure conditionnelle est dite à choix lorsque le traitement dépend de la valeur que peut prendre le sélecteur. Ce sélecteur doit être du type scalaire (entier ou caractère);

Une structure conditionnelle est dite généralisée lorsqu'elle permet de résoudre les problèmes comportant plus de deux traitements en fonction des conditions.

L'exécution d'un traitement entraîne automatiquement la non exécution des autres (alternative multiple).

L'alternative réduite: Elle ne s'exécute que si la condition est satisfaisante.

syntaxe : si (condition) alors (instruction)

Exemple : Si $x \neq 0$ alors $s \leftarrow 0$

L'alternative complète : Dans ce cas, une instruction est prévue pour chaque valeur logique de condition.

syntaxe : si (condition) alors (instruction) sinon (instruction)

Exemple : Si $x \neq 0$ alors $s \leftarrow b$ sinon $s \leftarrow 0$

Exercice : racine carré d'un nombre

- Algorithme Squart
 - Var x, y: réel
- Début
 - Ecrire ("enter un nombre");
 - Lire(x);
 - Si $x \geq 0$;
 - $y \leftarrow \text{sqrt}(x)$;
 - Ecrire (y);
 - Sinon
 - Ecrire ("pas de racine réelle");
 - Finsi
- Fin

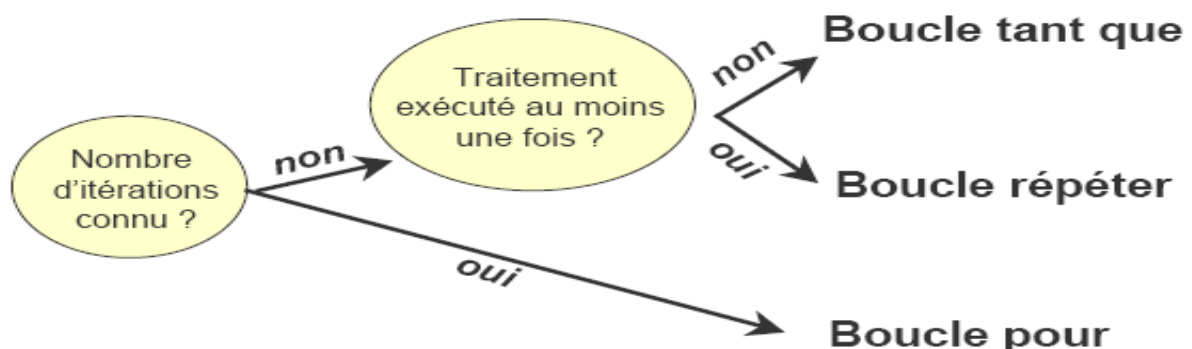
STRUCTURES ITÉRATIVES OU BOUCLES

Définition

Une **itération** ou boucle est une séquence d'instructions destinée à s'exécuter plusieurs fois.

Un **cycle** est un tour de la boucle.

On distingue deux types d'itérations: itérative complète (pour) et itérations à condition d'arrêt (tant que, répéter).



- La boucle **pour** : le nombre de répétition est connu. Sa syntaxe est :

Pour variable **allant** de valeur_initiale à valeur_finale **faire**

Action1 ;

Action2 ;

.....

FinPour

Exemple calculer la puissance d'un nombre

Algorithme puissance

Var n, i : entier; x, p: réel ;

Début

P ← 1;

Pour i ← 1 à n faire

p ← p*x ;

finPour

Fin

- La boucle **Tant que** : le nombre de répétition n'est pas connu. On teste une condition à l'entrée de la boucle avant d'exécuter les instructions : c'est une boucle conditionnelle. Sa syntaxe est :

Tant que (condition vraie) **Faire**

Action1 ;

Action2 ;

.....

FinTantque

- La boucle **Répéter** : ici, on exécute au moins une fois le bloc d'instructions avant de tester la condition. Sa syntaxe est :

Répéter

Action1 ;

Action2 ;

.....

Jusqu'à (condition) ;

Exemple: un algorithme qui calcule la moyenne des notes.

Algorithme Moyenne

Var s: réel; note: entier ;

Début

s ← 0;

i ← 1;

Répéter

Ecrire ("donner la note n°:", i);

Lire (note);

s ← s+note;

```
i ← i+1;  
Jusqu'à i >= 10  
Ecrire ('la moyenne des 10 notes est:', s/10);  
Fin
```

SITUATION D'INTEGRATION

On souhaite écrire un algorithme permettant d'afficher la liste de tous les nombres premiers inférieurs à n.

1. A quelle structure algorithmique correspond t-il ? Donner sa syntaxe.
2. Enumer les autres structures utilisés en algorithme.

REINVESTISSEMENT

Écrire un algorithme permettant d'afficher les entiers de 1 à n en utilisant : la boucle Pour et la boucle Tant que.

OBJECTIFS PEDAGOGIQUES :

- ✓ Exécuter un algorithme ayant une structure alternative ;
- ✓ Exécuter un algorithme simple ;
- ✓ Exécuter des algorithmes itératifs

Contrôle de prérequis :

1. Donner la structure minimale d'un algorithme
2. Utiliser les instructions d'écriture, d'affichage et d'affectation pour écrire les algorithmes.
3. Identifier et utiliser les structures de contrôle
4. Connaître les opérations mathématiques et logiques

SITUATION PROBLEME :

Votre amie Diana a écrit les algorithmes qui aideront sa maman à faire ses petits calculs au marché. N'étant pas très sûr de ce qu'elle a écrit, Diana vous fait appel dans le but de l'aider à tester ses algorithmes pour ne pas induire sa maman en erreurs lors de l'utilisation de ces algorithmes.

Consignes :

1. Que veut dire exécuter un algorithme ? (**Réponse attendue** : c'est faire dérouler chacune des instructions constituant cet algorithme dans l'ordre précis afin d'obtenir un résultat)
2. Que doit-on savoir lors de l'exécution d'un algorithme ? (**Réponse attendue** : l'ordre de priorité des opérateurs qui se trouve dans cet algorithme.).

RESUME

Un algorithme étant une suite d'instructions rangées dans un certain ordre, **exécuter un algorithme** revient donc à exécuter une à une chacune des instructions qui constituent cet algorithme et surtout en respectant l'ordre dans lequel ces instructions sont écrites, dans le but de savoir le résultat que produit cet algorithme.

Pour exécuter un algorithme, il faut :

- ✓ **Distinguer une instruction** : une instruction dans un algorithme se trouve dans le corps de l'algorithme entre Début et fin et se terminant toujours par un point-virgule (;)
- ✓ **Connaître l'ordre de priorité des opérateurs** pour effectuer les opérations arithmétiques.
- ✓ **Donner les valeurs aux variables** déclarés dans cet algorithme et remplacer chaque variable par sa valeur au cours de l'exécution puis effectuer les opérations.
- ✓ **Noter le contenu des variables** (Utiliser un tableau si nécessaire) à chaque étape d'exécution de l'algorithme.

- ✓ **Savoir quelle est la variable de sortie** et donner son contenu après l'exécution.

Exemple 1 : Exécution d'un algorithme itératif

Exécuter l'algorithme suivant avec $a=-4$ et $b=-2$

- (1). **Algorithme Plusgrand**
- (2). **Variable** a, b, pg : réel ;
- (3). Début
- (4). Ecrire ('entrer les deux valeurs') ;
- (5). Lire (a, b) ;
- (6). Si $a > b$ alors
- (7). $Pg \leftarrow a$;
- (8). Sinon
- (9). $Pg \leftarrow b$;
- (10). FinSi
- (11). Ecrire ('le plus grand nombre est :', pg) ;
- (12). Fin

Solution

Ligne	Instruction	Variables			Valeur de la condition	Ligne suivante
		A	B	pg		
5	Lire (a,b)	-4	-2	X		6
6	Si $a > b$	-4	-2	X	faux	7
7	$pg \leftarrow a$	-4	-2	X		8
8	Sinon	-4	-2	X	vrai	9
9	$pg \leftarrow b$	-4	-2	-2	vrai	11
11	Ecrire (pg)	-4	-2	-2	vrai	Fin

Cette algorithme retourne donc la valeur **-2** .

Exemple 2 : Algorithme itérative (cas de la boucle tant que)

Exécuter cet algorithme pour $a=2$, $b=3$

- (1).Algorithme BABATA
- (2).Var a, b, i : entier ;
- (3).S, p : réel ;
- (4).Début
- (5).Ecrire (entrer deux nombres) ;
- (6).Lire (a, b) ;
- (7). $S \leftarrow 0$; $p \leftarrow 0$; $i \leftarrow 1$;
- (8).Tantque ($i \leq b$) faire
- (9). $s \leftarrow s+i$; $p \leftarrow p+a$;
- (10). $i \leftarrow i+1$;

- (11). Fintanque
 (12). Ecrire (S; P);
 (13). Fin

Solution

	Test	a	b	S	P	i
6-7	$i \leq b$	2	3	0	0	1
8-10	$1 < 3$	2	3	$1+0=1$	$0+2=2$	$1+1=2$
8-10	$2 < 3$	2	3	$1+2=3$	$2+2=4$	$2+1=3$
8-10	$3=3$	2	3	$3+3=6$	$4+2=6$	$3+1=4$
8-10	$4 > 3$	/	/			
12	/	/	/	6	6	/

Les valeurs de retour sont : S=6 et P=6

Exemple 3 : Algorithme itérative (Cas de la boucle Pour)

Tester cet algorithme avec n=4

Algorithme Sommenombre

Var n, s, i : entier ;

Début

$i \leftarrow 0$; $S \leftarrow 0$;

Ecrire ('entrer la valeur de n') ;

Lire(n) ;

Pour $i \leftarrow 1$ à n

$s \leftarrow s+i$;

$i \leftarrow i+1$;

FinPour

Ecrire (n, i, s) ;

Fin

Solution

i	n	S
0	4	0
1	4	$0+1=1$
2	4	$1+2=3$
3	4	$3+3=6$
4	4	$6+4=10$
4	4	10

Les valeurs de retour sont i=4, n=4 et s=10

SITUATION D'INTEGRATION

Soit l'algorithme ci-dessous :

Algorithme : Square

a, b, c, out : Entier

Début

- 1) Lire (a)
- 2) $b \leftarrow a - 1$
- 3) $c \leftarrow b \times b$
- 4) $b \leftarrow 2 \times a - 1$
- 5) $out \leftarrow c + b$
- 6) Afficher (out)

Fin

Recopier puis compléter le tableau suivant :

Lignes	a	b	c	out
1	12			
2				
3				
4				
5				
6				

REINVESTISSEMENT

Soit l'algorithme ci-dessous :

Algorithme Calcul

Var R, i, n : entier

Debut

Ecrire ('Donner un entier >0');

Lire (n);

$R \leftarrow 1$

Pour i $\leftarrow$ 1 à n faire

$R \leftarrow R \times i$

Finpour;

Ecrire (R);

Fin

Exécuter à la main cet algorithme avec les valeurs n=12 et n=6

15

UNITE D'ENSEIGNEMENT 15 : ORGANIGRAMME

OBJECTIFS PEDAGOGIQUES :

- ✓ Construire un organigramme ;

CONTROLE DE PREREQUIS

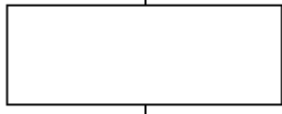
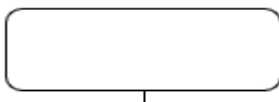
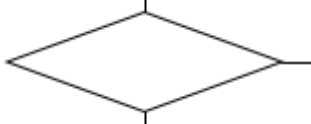
1. Connaître les différentes structures algorithmes ;
2. Connaître les opérations mathématiques et logiques

SITUATION PROBLEME

Votre petit frère a écrit un algorithme pour aider votre papa à effectuer ses calculs. Pour bien expliquer son algorithme à votre papa, il souhaite représenter cela graphiquement à l'aide des symboles normalisés. Etant donné qu'il ne connaît pas grand-chose à propos, il sollicite votre aide dans le but de l'aider à résoudre ce problème.

Consignes

1. Quels sont les différents langages de représentation d'un algorithme ? (**Réponse attendue** : Langage de description des algorithmes, et l'organigramme).
2. Qu'est-ce qu'un algorithme ? (**Réponse attendue** : un algorithme est la représentation schématisée d'un algorithme).
3. Quelle partie de l'algorithme représente-t-on graphiquement ? (**Réponse attendue** : le corps de l'algorithme : commençant par le mot clé Début, suivi de toutes les instructions contenues dans le corps de l'algorithme et se terminant par le mot clé Fin).
4. Donner le symbole utilisé en algorithme pour représenter les instructions et les mots clés suivantes : lecture/écriture, début/fin, affectation et une condition.

Instruction / mot clé	Symbole	Instruction / mot clé	Symbole
Lecture/écriture		Affectation	
Début/ fin		Condition	

RESUME

A part le LDA (langage de description des algorithmes), on peut aussi représenter un algorithme à l'aide des symboles normalisés. Cette représentation est appelée organigramme ou algorithme.

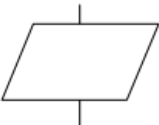
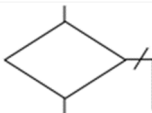
Un **Organigramme** : est la représentation graphique de l'algorithme. Il permet de représenter chaque opération élémentaire au moyen d'un symbole graphique

normalisé. Un organigramme bien représenté doit être fléché et fermé, compris entre un début et une fin, et doit permettre de suivre facilement l'ordre d'exécution des règles de résolution du problème étudié.

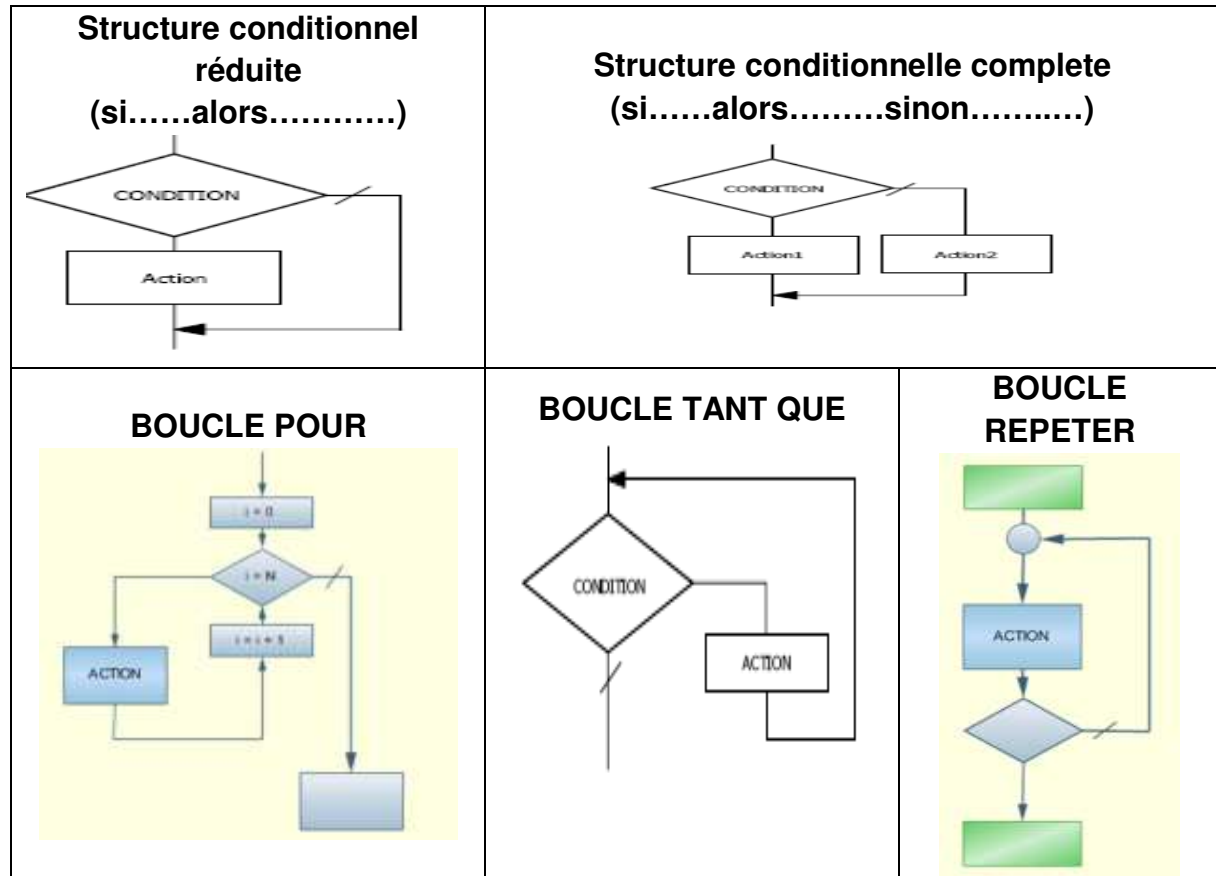
NB : Le sens général des lignes de liaison doit être :

- ✓ De haut en bas
- ✓ De gauche à droite
- ✓ Lorsque le sens général ne peut pas être respecté, des pointes de flèches à cheval sur la ligne indiquent le sens utilisé.

Pour construire un organigramme, on utilise des symboles normalisés donnés ci-dessous :

Symbole	Désignation	Symbole	Désignation
	Opération pour laquelle il n'existe aucune symbole normalisé (traitement)		Entrée-Sortie
	Début, fin ou interruption d'un organigramme		Conditions variables impliquant un choix parmi plusieurs

ORGANIGRAMME DES STRCUTURES



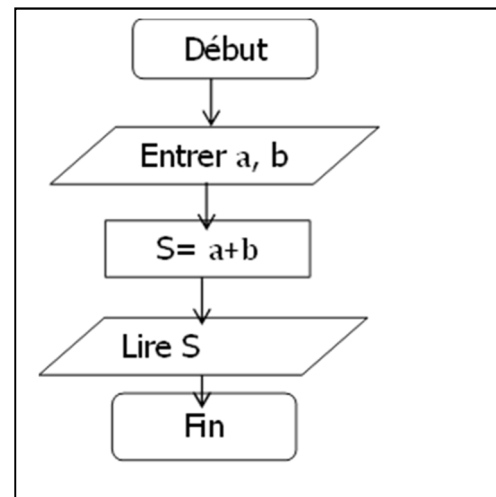
REGLES DE CONSTRUCTION DES ALGORIGRAMMES

L'organigramme doit être clair, aéré et lisible.

- L'organigramme doit être construit de manière à ce que la lecture se déroule verticalement ;
- Les sorties des tests ne doivent pas être inversées au cours d'un même organigramme. ;
- Les lignes de liaison entre les symboles ne doivent pas se couper ;
- Une ligne de liaison doit toujours arriver sur le haut et au centre d'un symbole ;
- Si l'organigramme semble être complexe, il faut le décomposer de façon à ce que ça soit simple ;
- Les commentaires sont généralement placés à droite de l'organigramme alors que les renvois de niveau sont à gauche

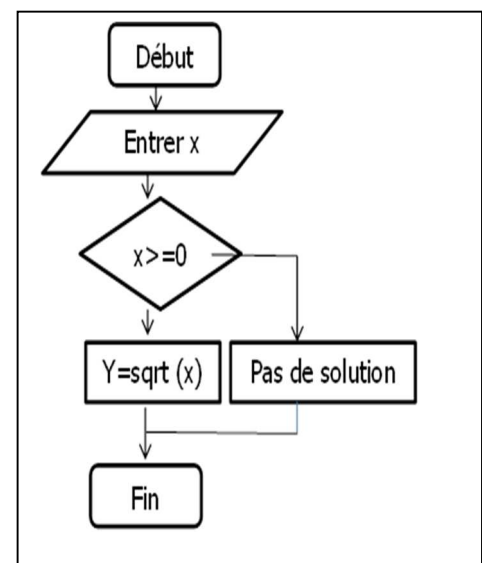
Exemple d'algorithme

- Algorithme somme
 - Variable a, b: réel
 - s: réel
- Début
 - Écrire ("entrer le premier nombre");
 - Lire (a);
 - Écrire ("entrer le deuxième nombre");
 - Lire (b);
 - $s \leftarrow a+b$;
 - Écrire (s);
- Fin



Exercice : racine carré d'un nombre

- Algorithme Squart
 - Var x, y: réel
- Début
 - Ecrire ("enter un nombre");
 - Lire(x);
 - Si $x \geq 0$;
 - $y \leftarrow \text{sqrt}(x)$;
 - Ecrire (y);
 - Sinon
 - Ecrire ("pas de racine réelle");
 - Finsi
- Fin



SITUATION D'INTEGRATION

On veut écrire un algorithme qui permet d'afficher le plus grand nombre parmi deux choisis et entrer par l'utilisateur.

- 1) Identifier la structure algorithmique correspondante
- 2) Ecrire l'algorithme complète resolvant ce problème.
- 3) Représenter l'organigramme de cet algorithme

REINVESTISSEMENT

Dessiner l'algorithme d'un algorithme qui calcule l'image d'un nombre x par une fonction du type $f(x) = ax+b$ (a et b donnés).